Project Documentation: Space Colony

Time allocation	0.5
Priority	1
Course	OOP
Deadline	April 20, 2026
Type	DO

Topic

I implemented implement an Android mobile game called Space Colony. The game allows players to create, train and send crew members on missions against a variety of threats.

Basic Functionality

The application includes the following basic features:

1. ✅Crew Management
    a. ✅Users can create different types of crew members (Pilot, Engineer, Medic, Scientist, Soldier).
    b. ✅Newly recruited crew members are placed in Quarters.
    c. ✅Users can move crew members to the ✅Simulator or ✅Mission Control.
2. Training System
    a. ✅Crew members gain experience points when trained in the Simulator.
    b. Experience increases skill power (e.g., if XP = 2, skill increases by 2 points).
3. Cooperative Mission System
    a. ✅Users can select two crew members to go on a mission.

b. ✅A threat is generated by the system with stats based on a scaling formula.

c. Missions follow a turn-based system:
   - ✅Crew members take turns acting against the threat.
   - ✅The threat retaliates after each crew member's action.
   - ✅The mission continues until the threat's energy drops to zero or both crew members are defeated.
   - ✅Surviving crew members gain experience points.
   - ~~Defeated crew members are removed from the program.~~

4. Crew Recovery

   a. ✅When a crew member returns to Quarters, its energy is fully restored but
   experience points remain.

Implemented extra features

Feature	Description	Points
Object-Oriented Code	The program is coded according to the object-oriented paradigm.	Required
Code in English	All code and comments must be in English. However, documentation and UI elements can be in Finnish.	Required
Android Compatibility	The program must work on various Android phones and be coded in Java using Android Studio.	Required
Basic Functionality	The program must implement all the features defined above so that users can interact with crew members.	Required
Documentation	(PDF or GitHub .md file) containing: class diagram, division of	Required

Feature	Description	Points
	work among team members, implemented features, and a general project description.	
Detailed Project Plan		+3
RecyclerView	Uses the RecyclerView component to list crew members and their details.	+1
Crew Images	Different crew specializations are visualized with unique images.	+1
Fragments	Use fragments meaningfully in the application.	+2
Mission Visualization	Adding visual effects including crew member images, giving the threat a random tint, lightsaber animation.	+2
Statistics	Track how each crew member has performed (number of missions, victories, training sessions, etc.).	+1
Statistics Visualization	Visualize game statistics in some way.	+2
No Death	Instead of dying, a defeated crew member is sent to Quarters and returns to initial stats after a penalty. The number of losses is shown in the individual charts.	+1
Colors	User can tint each crew member using a color picker scale, to enhance	+1

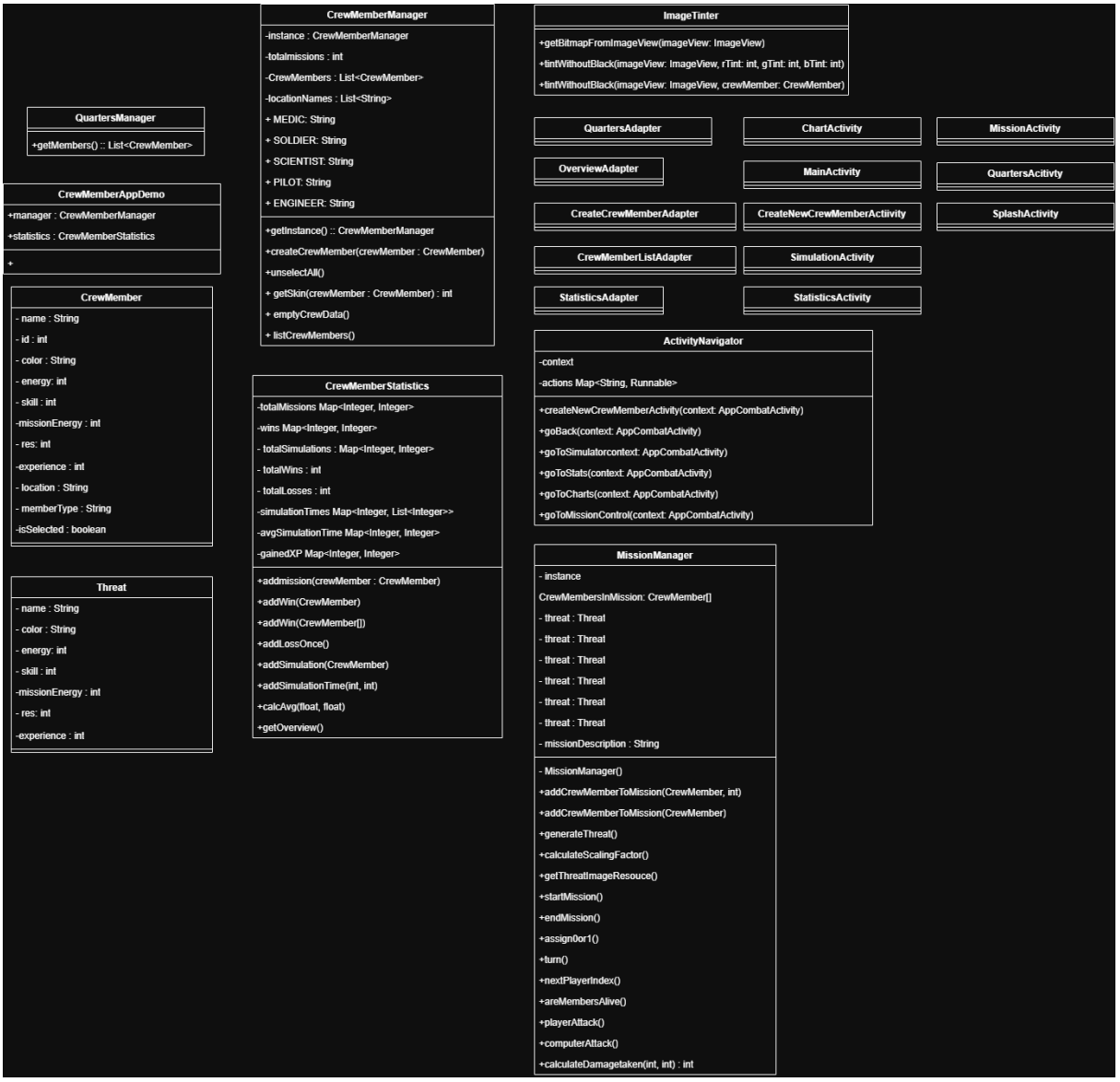
Feature	Description	Points
	individualism among crew members	
Custom loading screen	Splash activity (Hero group picture generate by ChatGPT).	+1
Randomness in Missions	Add some randomness to missions by randomizing what threat is generated and its ability stats.	+1
Total:	Required + Bonus points + Project Plan = 13 + 11 + 3	27/25 → 25/25

Team members and distribution of work

Lilla Bagossi — 002592176

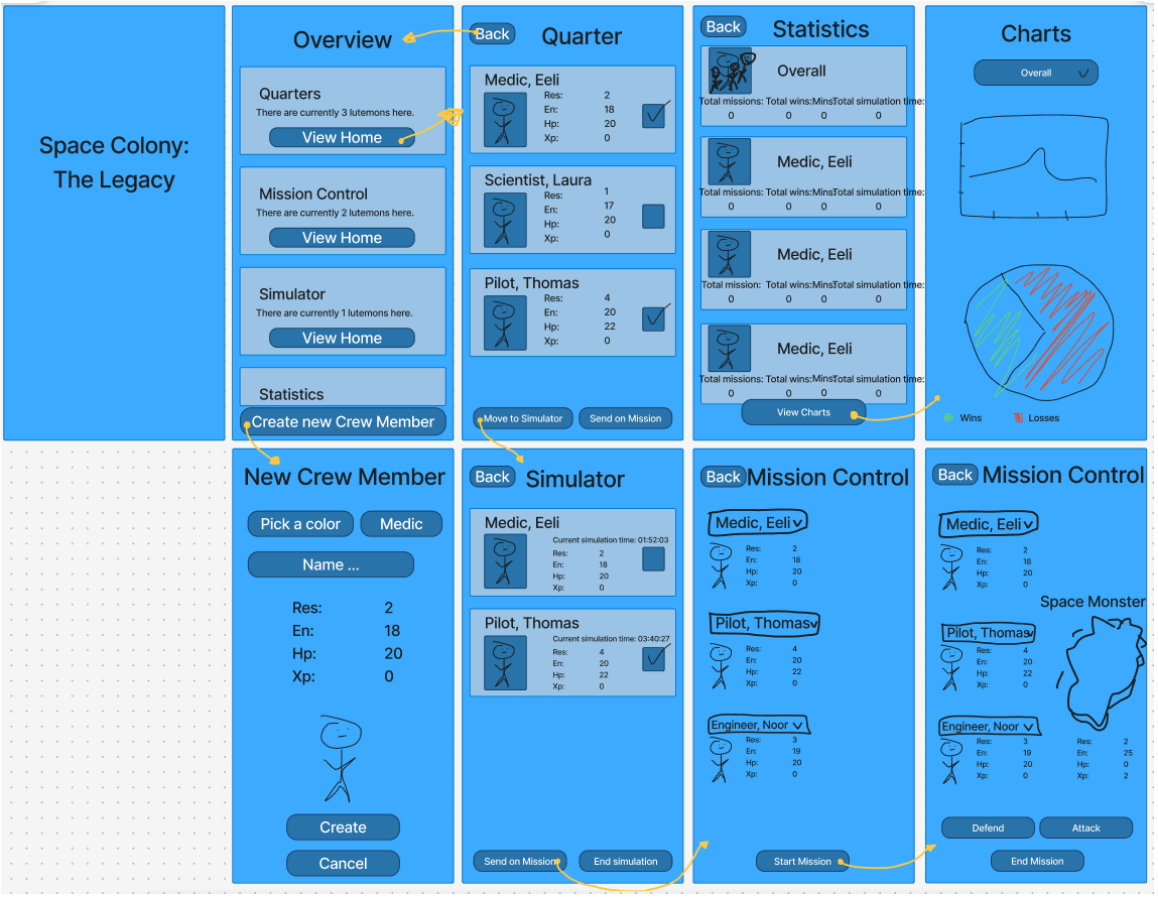
- Planning
- Designing UI
- Creating and refining UML diagrams
- Coding
- Documentation

UML Class Diagrams



Preliminary User Interface

Low-fidelity prototype made with FigJam. An additional video, explanaining the views, can be found here.



There are 7 *interactive* windows. Each window inherits the same header (with heading and back button) except the “Create New Crew Member” window. RecyclerViews are used to list the:

- locations (Quarters, Mission, Simulator and Statistics) in the Overview. Each item has a button, which brings the user to the respective location.
- Crew Members currently residing in Quarters. Each card displays a Crew Member’s information (type, name, then resilience, energy, health points and experience points). There is a tick next to each Crew Member, which allows the user to select Crew Members of his/her liking to send on a Mission or to the Simulator.
- Crew Members currently residing in the Simulator. A timer displays how long each Crew Member has been training. The user is able to send any Member from here on a Mission or back to Quarters, which automatically ends the simulation.
- all Crew Members that exist, with their respective statistics including wins, losses and times spend in simulation.

Dropdown lists are used:

- In the “Create New Crew Member” fragment to choose member type of one’s liking.
- In the Mission Control, to choose which (if any) Members should be added to a new Mission.
- In the Charts, to select which Member’s visualized statistics to display.

Battle logic with 2 players:

- The enemy acts “automatically” based on simple programmed “AI” logic. There is a few seconds of “waiting time”.
- Whichever party is in action is obvious by animation: There is a lightsaber, and whichever direction it points from shows who’s turn it is.

Threat generation logic:

Soldier → Asteroid storm, alien attack

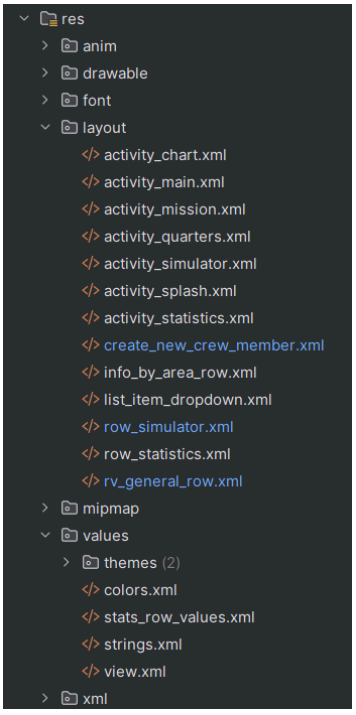
Medic → Fire in the kitchen

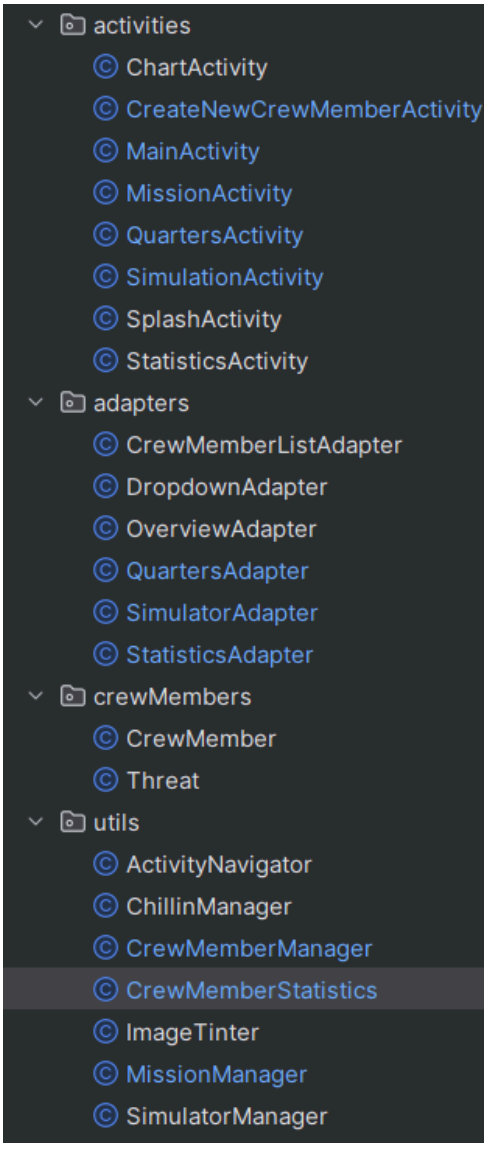
Engineer → Fuel leakage

Scientist → Solar flares

Pilot → Fuel leakage, asteroid storm

Directory structure

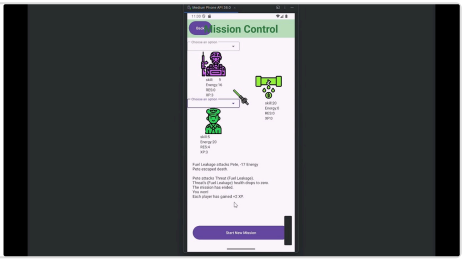




Demonstration Video

OOP Project Video

https://www.youtube.com/watch?v=77O5Zd7eyJ0



Source Code

https://github.com/lillabbbbbb/space_colony

AI declaration statement

No AI was used in the writing of the project description. In the project implementation the use of AI (ChatGPT) is marked with code comments.